1. Introducción

En este proyecto he desarrollado el paquete `turtle_control` para mejorar el control y manejo del simulador `turtlesim` en **ROS 2 (Jazzy)**. La idea principal es dejar atrás el teleoperador básico que viene por defecto (que bloquea la terminal y solo permite avanzar o girar de forma rígida y por separado), sustituyéndolo por un control mucho más natural donde la tortuga puede **avanzar y girar a la vez** con W, S, A y D para trazar curvas y círculos fluidos. Además, he conectado el nodo directamente a los **servicios** de Turtlesim para poder activar o pausar el dibujo con la barra espaciadora, borrar la pantalla con la tecla C y recolocar la tortuga en el centro mirando hacia arriba con la tecla R, permitiendo también cambiar la velocidad o los logs en caliente mientras el nodo está funcionando.

2. Funciones Implementadas para el Control del Robot

La solución se encapsula en el nodo `TeleopTurtle` (dentro de `turtle_control/teleop_turtle.py`), desarrollado en Python utilizando la librería oficial `rclpy`. Se articulan las siguientes funciones y componentes técnicos:

2.1. Captura No Bloqueante y Movimiento Simultáneo (Teclas W, S, A, D)

Para superar la limitación del buffer bloqueante de consola habitual en los nodos de teleoperación, se integra la librería `pynput.keyboard.Listener` en un hilo secundario desacoplado (*daemon thread*), evitando pausar el ciclo de eventos principal de ROS:

- **Conjunto de teclas concurrentes:** El nodo gestiona las teclas activas en la estructura `self.pressed_keys = set()`. En el callback `on_key_press` se añade la tecla pulsada (`w`, `s`, `a`, `d`), y en `on_key_release` se descarta de forma inmediata.
- **Cálculo cinemático diferencial a 30 Hz (`timer_callback`):** Un temporizador periódico publica a 30 Hz en `/turtle1/cmd_vel` el mensaje `geometry_msgs/msg/Twist` sumando los componentes algebraicos:

  ```
  linear = (self.speed if 'w' in self.pressed_keys else 0.0) - (self.speed if 's' in self.pressed_keys else 0
  angular = (self.angular_speed if 'a' in self.pressed_keys else 0.0) - (self.angular_speed if 'd' in self.pre
  twist.linear.x, twist.angular.z = float(linear), float(angular)
  self.cmd_vel_pub.publish(twist)
  ```

  Al presionar a la vez `W` y `A` (o `W` y `D`), el simulador recibe tanto velocidad lineal en X como velocidad angular en Z, describiendo curvas continuas y trayectorias suaves.

2.2. Invocación Asíncrona de Servicios de Turtlesim

Para no pausar la publicación periódica de velocidad, todas las peticiones a servicios se envían de forma asíncrona mediante el método `call_async()`:

- **Control de Pluma (ESPACIO → `/turtle1/set_pen`):** La función `toggle_pen()` conmuta la variable booleana `self.pen_enabled` y envía una petición `SetPen.Request` fijando `req.off = 0` (activar pintado) o `req.off = 1` (desactivar pintado), permitiendo circular sin trazar línea cuando se desee.
- **Limpieza de Pantalla (C → `/clear`):** La función `call_clear()` envía una solicitud de tipo `std_srvs/srv/Empty`, borrando las trayectorias dibujadas sobre el lienzo sin alterar las coordenadas actuales del robot.
- **Reinicio Central Cenital (R → `/turtle1/teleport_absolute`):** El comando estándar `/reset` de Turtlesim orienta la tortuga hacia la derecha (0 rad). Para acatar estrictamente la especificación de diseño (*"al centro de la pantalla y mirando hacia arriba"*), la función `call_reset_center_up()` utiliza `TeleportAbsolute.Request` fijando `x=5.5444`, `y=5.5444` y `theta = π / 2` radianes (90° exactos).

2.3. Parámetros Dinámicos en Tiempo de Ejecución

El nodo declara formalmente: `speed` (def. 2.0), `angular_speed` (def. 2.0) y `log_level` (def. 'INFO'). Mediante `self.add_on_set_parameters_callback(self.on_parameter_change)`, se capturan en tiempo real los cambios enviados por consola con `ros2 param set`, actualizando al vuelo las velocidades cinemáticas o la severidad del log sin reiniciar el nodo.

2.4. Automatización con Lanzador (Launch File)

El archivo `launch/turtlesim_teleop.launch.py` declara los argumentos de configuración (`DeclareLaunchArgument`) e inicializa coordinadamente tanto `turtlesim_node` como `teleop_turtle`, configurando `emulate_tty=True` para mantener la captura directa por consola.

3. Interfaces y Comunicaciones en el Grafo ROS 2

Canal / Mecanismo	Nombre del Recurso	Tipo de Interfaz	Función y Propósito
Topic (Publisher)	`/turtle1/cmd_vel`	`geometry_msgs/msg/Twist`	Publicación continua (30 Hz) de velocidad lineal X y angular Z.
Service Client	`/turtle1/set_pen`	`turtlesim/srv/SetPen`	Activa o desactiva el trazo de pintura (tecla `ESPACIO`).
Service Client	`/clear`	`std_srvs/srv/Empty`	Borra todas las trayectorias dibujadas (tecla `C`).
Service Client	`/turtle1/teleport_absolute`	`turtlesim/srv/TeleportAbsolute`	Posiciona en centro `(5.54, 5.54)` mirando al norte `θ = π/2` (tecla `R`).
Parámetros	`speed` / `angular_speed`	`Double`	Control en caliente de la velocidad de traslación y giro.
Parámetro	`log_level`	`String`	Ajuste dinámico del nivel de log (DEBUG, INFO, WARN, ERROR).

4. Guía de Compilación y Comandos de Ejecución

1. Compilación del espacio de trabajo:

```
cd ~/ROS/practica\ 1    # o dentro del espacio de trabajo ws
colcon build --packages-select turtle_control
source install/setup.bash
```

2. Ejecución mediante el lanzador integrado:

```
# Lanzamiento unificado con parámetros por defecto:
ros2 launch turtle_control turtlesim_teleop.launch.py

# Lanzamiento alternativo modificando parámetros iniciales:
ros2 launch turtle_control turtlesim_teleop.launch.py speed:=3.5 log_level:=DEBUG
```

3. Modificación interactiva de parámetros en caliente:

```
ros2 param set /teleop_turtle speed 5.0        # Incrementa la velocidad lineal en caliente
ros2 param set /teleop_turtle log_level DEBUG   # Modifica la severidad de log del nodo
```

5. Matriz de Cumplimiento de Requisitos y Especificaciones Técnicas

Requisito / Especificación Técnica	Mecanismo Técnico Implementado
Movimiento continuo con W, S, A, D	Mapeo de teclas sobre vector `geometry_msgs/Twist`.
Giro y avance simultáneo (curvas continuas)	Conjunto concurrente con `pynput` y publicación periódica a 30 Hz.
Tecla SPACE (habilitar/deshabilitar trazo)	Cliente asíncrono para `/turtle1/set_pen` con flag conmutable `off`.
Tecla C (borrado de dibujo)	Cliente asíncrono para servicio nativo `/clear`.
Tecla R (centro y mirando hacia arriba)	Teletransporte a `x=5.544, y=5.544, θ=π/2` con `/turtle1/teleport_absolute`.
Lanzador dedicado (Launch File)	`turtlesim_teleop.launch.py` lanza conjuntamente simulador y control.
Parámetro de nivel de logging	Parámetro reactivo `log_level` con callback dinámico de severidad.
Parámetro de velocidad en tiempo real	Parámetro `speed` reactivo mediante callback ante `ros2 param set`.